

Tiered vs. Uniform Model Allocation in a Multi-Agent Travel-Planning System

Tiered vs. Uniform Model Allocation in a Multi-Agent Travel-Planning

System: A Pilot Study

Draft status: this reports one pilot run (RQ2 only, n=13 scripted conversations) against a system implemented for this purpose. It is not a peer-reviewed or statistically powered result — see Limitations. RQ1/RQ3/RQ4/RQ5 in RESEARCH-DIRECTION.md are proposed but not yet run; they are described here as future work only.

Abstract

We describe a LangGraph-based multi-agent travel-planning assistant built as an extension of an existing single-node RAG customer-service bot, and use it to pilot one experimental design question: does allocating a larger “orchestrator” model to routing/synthesis and a smaller “worker” model to narrow retrieval tasks (tiered allocation) change task success, routing behavior, or cost relative to using one model size uniformly? Across 13 scripted multi-turn conversations and three model-allocation conditions (tiered: gpt-4o orchestrator / gpt-4o-mini worker; uniform-large: gpt-4o everywhere; uniform-small: gpt-4o-mini everywhere), all three conditions passed the identical 9/13 tasks, with the same four failures recurring in every condition. We show these four failures are structural implementation defects independent of model tier, not evidence that tier doesn’t matter — and report one significant tier-dependent difference we did

find: the uniform-small condition routed to the retrieval agent less than half as often as the other two conditions while reaching the same pass rate, a routing-efficiency effect distinct from accuracy.

1. Introduction

Travel planning is a compositional task: it requires retrieving and reconciling information from independently-shaped sources (flight and hotel availability/price, destination weather and entry requirements, a fixed budget, and a user-specified but often incompletely stated set of constraints) into a single itinerary that satisfies all of them at once. This makes it a natural, if unglamorous, instance of the broader problem of using LLM-based agents for multi-constraint, multi-tool planning under partial information. Unlike synthetic agent benchmarks, TravelPlanner (Xie et al.) is a benchmark built specifically around this domain, which makes travel planning a reasonable vehicle for studying multi-agent orchestration design choices rather than only building a demo.

The system studied here (ICS Travel Agent) began as a single-node LangGraph RAG assistant and was extended into a supervisor/specialist multi-agent graph: a supervisor routes each turn to one of seven specialists (`flight`, `hotel`, `local_info`, `itinerary`, `budget`, `rag`, `responder`) via LangGraph Command objects, accumulating slot-filled trip constraints across turns until enough is known to produce an itinerary and budget summary. Full architecture is documented in [../01-architecture.md](#) and [../02-agent-design.md](#); this paper assumes that design and focuses on one axis of it.

1.1 Research question

This system exposes several design axes that could be ablated (routing mechanism, model allocation, node determinism, hop budget, routing grounding — enumerated as RQ1-RQ5 in [RESEARCH-DIRECTION.md](#)). This paper reports a pilot on one of them:

RQ2: What is the accuracy/cost relationship between tiered model allocation (large orchestrator, small worker) and a uniform-model baseline, on a fixed task set?

The other four axes are described in Section 6 as future work; no data is reported for them here.

2. System design (relevant subset)

- **Topology:** hub-and-spoke supervisor with Command-based dynamic handoff, not a statically declared conditional-edge graph (see [../02-agent-design.md](#)).
- **Model allocation:** `flight`, `hotel`, `local_info`, and `budget` are pure deterministic tool calls with no LLM step. `supervisor`, `itinerary`, and `responder` use the orchestrator-tier model

(gpt-4o); rag uses the worker-tier model (gpt-4o-mini) — see [../03-llm-infra-decision.md](#). Both model names are configured once in `backend/config.py` and read fresh on every call, which is what makes the ablation in this paper possible without touching agent code.

- **Hop budget:** each turn is capped at `MAX_HOPS_PER_TURN = 6` supervisor-to-specialist transitions, an unvalidated fixed constant (RQ4).
- **Data sources:** flight/hotel results are deterministic mocks seeded by query parameters (not live market data); visa data covers a small fixed set of corridors; weather and RAG retrieval hit real services/indexes. This scopes every result in this paper to *orchestration logic*, not real-world booking correctness.

3. Method

3.1 Task set

13 scripted multi-turn conversations (`backend/pilot/task_set.py`), stratified along the axes in [EVALUATION-PROTOCOL.md](#) section 1: slot completeness (full vs. under-specified/multi-turn), destination count (single vs. multi-city up to 5), constraint tightness (comfortable vs. over-budget), and domain mix (pure trip-planning vs. trip-planning mixed with the original RAG product-knowledge-base capability). Two tasks (T6/T7) test the rag agent directly against real, verified content from the underlying PDF corpus (Cambricon MLU370 accelerator manuals — the corpus is hardware spec sheets, not travel content, since the rag agent is a preserved continuation of the original single-node bot’s function, not a travel-domain RAG index). Full design rationale and two known defects surfaced while authoring the task set are documented in [PILOT-TASK-SET.md](#).

Each task’s `assertions(final_state)` scores the graph’s full final `TravelState` deterministically (field presence/shape, not exact string match, since destination strings are LLM-extracted free text and not canonicalized).

3.2 Conditions

Three model-allocation conditions, switched by monkeypatching `config.ORCHESTRATOR_MODEL/config.WORKER_MODEL` before each task run (`backend/pilot/harness.py`):

Condition	Orchestrator-tier calls use	Worker-tier calls use
tiered (current production config)	gpt-4o	gpt-4o-mini
uniform_orchestrator	gpt-4o	gpt-4o (same model)

Condition	Orchestrator-tier calls use	Worker-tier calls use
uniform_worker	gpt-4o-mini (same model)	gpt-4o-mini

All 13 tasks were run through `graph.invoke` directly (bypassing the HTTP layer’s reply-string-only return) under each condition, so scoring has access to the complete final state rather than just the user-facing reply text.

3.3 Metrics

Per task: pass/fail against its assertions. Per turn: hop count (`len(agent_scratchpad)`), orchestrator-tier vs. worker-tier call count (by which agent, per the table in Section 2, was visited), and wall-clock elapsed time. These follow the metric definitions in [./08-evaluation-methodology.md](#).

4. Results

All three conditions passed the identical 9 of 13 tasks:

Condition	Pass rate	Total hops (13 tasks)	Orchestrator-tier calls	Worker-tier (rag) calls	Total elapsed
Tiered	9/13	99	61	9	282.8s
Uniform-large	9/13	97	60	9	273.6s
Uniform-small	9/13	93	58	4	286.3s

4.1 Failures are structural, not model-dependent

The same four tasks — T1, T5, T8, T9 — failed in every condition. Because model tier did not change which tasks passed, we can localize each failure to a specific implementation defect rather than a capability gap:

- **T1** (`local_info_results` empty, `budget_summary` missing): the same destination string is passed to both a geocoding-based weather lookup and an ISO-code-keyed visa lookup, which structurally cannot both succeed from one naturally-phrased string.
- **T5** (`budget_summary` missing on a 3-city trip): budget aggregation always selects a single cheapest flight and hotel offer regardless of destination count, so it never produces a summary that accounts for a multi-city itinerary.

- **T8** (knowledge_context missing): the supervisor never routes a domain-mix turn (a request touching both trip-planning and the RAG knowledge base) to rag on its first turn — a routing miss, not a budget or capability issue.
- **T9** (agent_scratchpad has 7 entries against a stated cap of 6): the hop budget is not enforced as a hard ceiling for a 5-city stress request.

All four recur identically whether the model doing the routing/synthesis work is gpt-4o or gpt-4o-mini, which is itself informative: it rules out “the small model isn’t good enough” as the explanation and points at the routing/aggregation code paths instead.

4.2 A tier-dependent difference that isn’t in the pass rate

uniform_worker made 4 calls to rag across all 13 tasks’ turns, against 9 in both other conditions — despite passing the same 9/13 tasks, including the two tasks (T6/T7) written specifically to exercise rag. The smaller model is routing to retrieval less readily in ambiguous cases even where it doesn’t change the final pass/fail outcome on this task set. This is a routing-efficiency effect, not an accuracy effect, and the task set as designed can’t distinguish “correctly avoiding an unnecessary RAG call” from “under-triggering RAG” — that requires ground-truth labels for whether RAG *should* have been consulted on each ambiguous turn, which this pilot did not collect.

Total hops (93-99) and elapsed time (273.6-286.3s) are within noise of each other across conditions at this sample size; no condition shows a cost/latency advantage large enough to be worth a claim.

5. Limitations

- **n=13, single run per condition, no significance test.** EVALUATION-PROTOCOL.md calls for 30-50 conversations per stratum and a two-proportion z-test or Fisher’s exact test for pass-rate comparisons; at n=13 with an identical 9/13 across all three conditions there is no proportion difference to test. Any claim beyond “no directional signal at this scale, except in routing efficiency” would overstate what this pilot supports.
- **A fixed 4/13 floor masks whatever the true tier effect on accuracy is.** Because the same four tasks fail regardless of condition, this pilot cannot rule out an accuracy difference that would only appear once those defects are fixed and the remaining 9-13 “passing” tasks are pushed harder (e.g. larger task set, adversarial phrasing).
- **Single model family, single provider.** Both tiers are OpenAI models behind one aggregator (per ../03-llm-infra-decision.md); cross-provider or cross-family generalization is untested. Flight/hotel data are deterministic mocks, not live market data — results speak to orchestration logic, not real-world booking correctness.

- **No human evaluation of itinerary quality.** All metrics here are structural/automatic (field presence, hop count, call count). Whether a human traveler would find the resulting itinerary useful is out of scope, per [../08-evaluation-methodology.md](#).

6. Future work

- Fix the four structural defects identified in Section 4.1 (or explicitly scope them as known limitations) before running EVALUATION-PROTOCOL.md at its intended 30-50-per-stratum scale — otherwise every condition shares the same artificial floor.
- Collect ground-truth “should RAG have been consulted” labels for ambiguous domain-mix turns to turn the Section 4.2 routing-frequency gap into a precision/recall measurement rather than a raw count.
- Run RQ1 (Command-based dynamic handoff vs. static conditional-edge routing), RQ3 (deterministic vs. LLM-in-the-loop tool nodes), RQ4 (hop-budget sweep across {3, 6, 10, unbounded}), and RQ5 (tool- computed multi-city routing vs. LLM-inferred ordering) — all designed but not yet run, per [RESEARCH-DIRECTION.md](#).
- A small human-subjects comparison against a single-agent ReAct-style baseline, per RESEARCH-DIRECTION.md’s suggested future work, once the automatic metrics above no longer have a known structural floor confounding them.

Related work (pointers, not yet verified for this draft)

See [RELATED-WORK.md](#) for the fuller list. Most directly relevant here: TravelPlanner (Xie et al.) as the closest existing benchmark and source of constraint-satisfaction scoring methodology worth adapting for a larger run; Anthropic’s *Building Effective Agents* (2024) for the orchestrator-worker pattern this system’s topology implements; ReAct (Yao et al.) as the natural single-agent baseline for a future RQ1-style comparison. Citation details (exact venue/year) are not independently re-verified in this draft — see RELATED-WORK.md’s own caveat before submission.

Reproduction

```
cd backend
python3 -m pilot.harness --condition all --out pilot/
results/<condition>.json
python3 -m pilot.score
```

Raw per-task results and logs: backend/pilot/results/
{tiered,uniform_orchestrator,uniform_worker}.
{json,log}. Full narrative log entry: RESEARCH-LOG.md.